ENGINEERING DESIGN REPORT

SYSTEM ARCHITECTURE-BASED ENGINEERING DESIGN FOR A DISTRIBUTED AUTOMOTIVE EMBEDDED CONTROL SYSTEM

ECA14 – EMBEDDED SYSTEMS

CO1 – SYSTEM ARCHITECTURE-BASED ENGINEERING DESIGN ASSIGNMENT

Submitted by

Chethrika Sri KM

Register Number: 192511112

Department: CSE

Submitted to

Instructor: Revathi G

Date of Submission: 02 November 2026

Academic Year 2026–2027

ABSTRACT

This report proposes a distributed embedded control architecture for a luxury automobile integrating engine management, electronic braking, safety monitoring, body control and infotainment. Dedicated electronic control units (ECUs) are connected through a segmented CAN/CAN FD network and a controlled central gateway. Automotive-grade microcontrollers are selected for deterministic control ECUs, while an application-class SoC is reserved for infotainment. The report compares processing alternatives, defines message priorities and communication paths, evaluates Classical CAN and CAN FD, models electronic braking as a finite-state machine, and examines timing, reliability and fault scenarios. A hybrid network is recommended: Classical CAN for compact, predictable control traffic where payloads are small, and CAN FD for selected higher-payload domains after compatibility and timing analysis. The proposed design is suitable for detailed engineering development, subject to quantitative bus-load analysis, safety lifecycle activities and vehicle-level validation.

Keywords: embedded systems, automotive ECU, CAN, CAN FD, real-time systems, fault tolerance, finite-state machine.

1. INTRODUCTION

Modern automobiles are networked cyber-physical systems. Sensors observe vehicle and driver conditions, ECUs process those inputs, and actuators perform control actions. Functions such as engine management and braking must respond within bounded time, while body electronics and infotainment may tolerate longer delays. A suitable architecture must therefore balance determinism, safety, bandwidth, cost, power and future expansion.

The purpose of this assignment is to design and justify a high-level networked embedded architecture for five automotive domains. The design follows the engineering sequence of requirements, alternatives, analysis, selection, implementation concept, verification and deployment decision. It is a conceptual architecture rather than a production-ready safety case; exact timing budgets, hardware part numbers and vehicle-specific safety targets must be finalized during detailed development.

2. STAGE 1 – SYSTEM REQUIREMENT ANALYSIS

2.1 Functional requirements

- Engine control: acquire crank/cam position, air/fuel and temperature measurements; calculate torque and fueling commands; monitor engine faults.
- Braking control: read wheel-speed and brake-pedal/pressure signals; provide anti-lock braking and brake-pressure control; report status and faults.
- Safety monitoring: monitor crash and restraint sensors, vehicle dynamics and ECU health; command restraint devices and hazard warnings when validated trigger conditions occur.
- Body control: operate doors, windows, lighting, wipers and access functions based on switches, sensors and vehicle state.
- Infotainment: provide display, audio, navigation and user-interface functions while exchanging only approved vehicle information through a gateway.
- Network management: identify messages, prioritize time-critical traffic, detect communication faults, support diagnostics and permit controlled software/configuration updates.

2.2 ECU roles and interfaces

ECU	Inputs / sensors	Processing and outputs	Criticality
Engine ECU	Crank/cam, throttle, airflow, temperature, pressure	Torque/fuel/spark calculations; injector and ignition drivers	High; powertrain control
Braking ECU	Wheel speeds, pedal position, pressure, yaw rate	ABS/traction logic; hydraulic modulator commands	Safety-critical
Safety ECU	Crash sensors, restraint status, vehicle state	Airbag/belt commands, hazard request, fault supervision	Safety-critical
Body-control ECU	Switches, door states, rain/light sensors	Lamps, locks, windows, wipers	Non-critical to safety, but availability relevant
Infotainment ECU	Touch/display, audio, navigation and approved vehicle data	Media, navigation, HMI and connected services	Non-safety-critical

2.3 Real-time communication requirements

Control messages require bounded latency, low jitter and predictable delivery. Braking and safety messages receive the highest bus priority. Engine torque/speed and vehicle-dynamics data are also time-sensitive. Body-control messages generally have moderate deadlines, while infotainment payloads are high-volume but less time-critical. The detailed design shall assign each message a period, deadline, maximum payload, identifier and worst-case transmission time.

Traffic class	Example	Illustrative period / deadline	Priority
Safety event	Crash / restraint trigger	Event-driven; deadline defined by safety concept (typically milliseconds)	Highest
Chassis control	Wheel speed / brake status	5–10 ms period; bounded few-ms network latency target	Very high
Powertrain	Engine speed / torque request	10–20 ms period; bounded latency	High
Body control	Door / lighting status	20–100 ms period	Medium
Infotainment	Metadata / diagnostics / display data	Best effort or application-specific	Low

These values are initial design targets for analysis, not certified requirements. They must be confirmed using vehicle dynamics, safety analysis and schedulability testing.

2.4 Processing, memory and I/O requirements

- Control ECUs need deterministic interrupt response, hardware timers, ADCs, PWM outputs, watchdogs, communication peripherals and sufficient CPU margin for worst-case execution.
- Flash stores program code, calibration and diagnostic data; RAM stores sensor samples, control state, communication buffers and fault records. EEPROM or emulated non-volatile memory may retain selected settings.
- Infotainment requires substantially more memory, graphics and multimedia processing, often with an operating system and storage. It must not share uncontrolled execution resources with safety control.
- I/O protection, signal conditioning, automotive voltage/transient protection and diagnostics are required at ECU boundaries.

2.5 Reliability, fault tolerance and network separation

The system shall detect sensor plausibility faults, ECU execution failures, bus-off conditions, corrupted frames, power-supply anomalies and gateway faults. A fault must lead to a defined safe or degraded operating state rather than uncontrolled actuation. Safety-critical and infotainment domains should be separated physically or logically using independent bus segments and a gateway with allow-listed messages, rate limits and diagnostics. Infotainment must never directly command braking, steering or restraint actuators.

2.6 Constraints and System Requirement Statement

Key constraints include automotive temperature and voltage ranges, electromagnetic compatibility, wiring cost and weight, limited ECU power, required startup time, software complexity, safety integrity targets, network bandwidth, protocol compatibility and long vehicle service life.

System Requirement Statement. The proposed system shall provide five coordinated ECU domains that acquire sensor data, execute deterministic local control, drive appropriate actuators and exchange authenticated-by-design, integrity-checked and prioritized messages over segmented automotive networks. Safety-critical control shall remain locally executable during loss of non-critical communication, faults shall be detected and contained, and the architecture shall support controlled expansion to higher-payload applications without compromising bounded response times.

3. STAGE 2 – PROCESSING ARCHITECTURE ANALYSIS AND SELECTION

3.1 Comparison of processing alternatives

Architecture	Strengths	Limitations	Automotive suitability
MCU	Deterministic interrupt handling; integrated timers, ADC, PWM and CAN; low power and cost	Less compute/memory than application processors; limited graphics and large OS workloads	Excellent for engine, braking, safety and body ECUs
MPU	High processing throughput; large address space; supports rich operating systems	External memory/peripherals often needed; more complex startup, power and timing control	Suitable for infotainment and gateway applications; less suitable for tight control loops without real-time design

DSP	Efficient signal processing, filtering and mathematical operations	Specialized toolchain and less general-purpose I/O integration; may require companion MCU	Useful for audio, radar/sensor processing or advanced signal workloads
SoC	Integrates CPU, GPU, accelerators, memory interfaces and peripherals	Higher complexity, power and thermal design; resource partitioning required	Strong fit for infotainment and advanced driver-assistance computing

3.2 Design Decision 1 – Selected processing architecture

I recommend a heterogeneous architecture: automotive-grade MCUs for the engine, braking, safety and body ECUs, and an MPU/SoC for infotainment. The control functions require predictable interrupt response, integrated analog/digital I/O, timer/PWM peripherals, watchdogs and low power. These are natural MCU workloads. Infotainment requires graphics, multimedia, large memory and a rich operating system, so an application-class SoC is more appropriate.

A single processor type for every ECU would either over-specify simple control nodes or under-serve infotainment. A heterogeneous design optimizes cost and determinism while containing complex software in the non-safety domain. The safety ECUs should use qualified automotive components and development processes appropriate to the assigned safety goals; the exact MCU family is selected only after timing, temperature, safety and lifecycle requirements are known.

4. STAGE 3 – NETWORKED SYSTEM ARCHITECTURE

4.1 Proposed high-level architecture

The design uses dedicated ECUs connected to a segmented network. A central gateway routes only necessary information between domains. The powertrain/chassis network carries deterministic control traffic; a body network carries comfort functions; infotainment is placed on a separate high-bandwidth domain and receives a restricted vehicle-data feed.

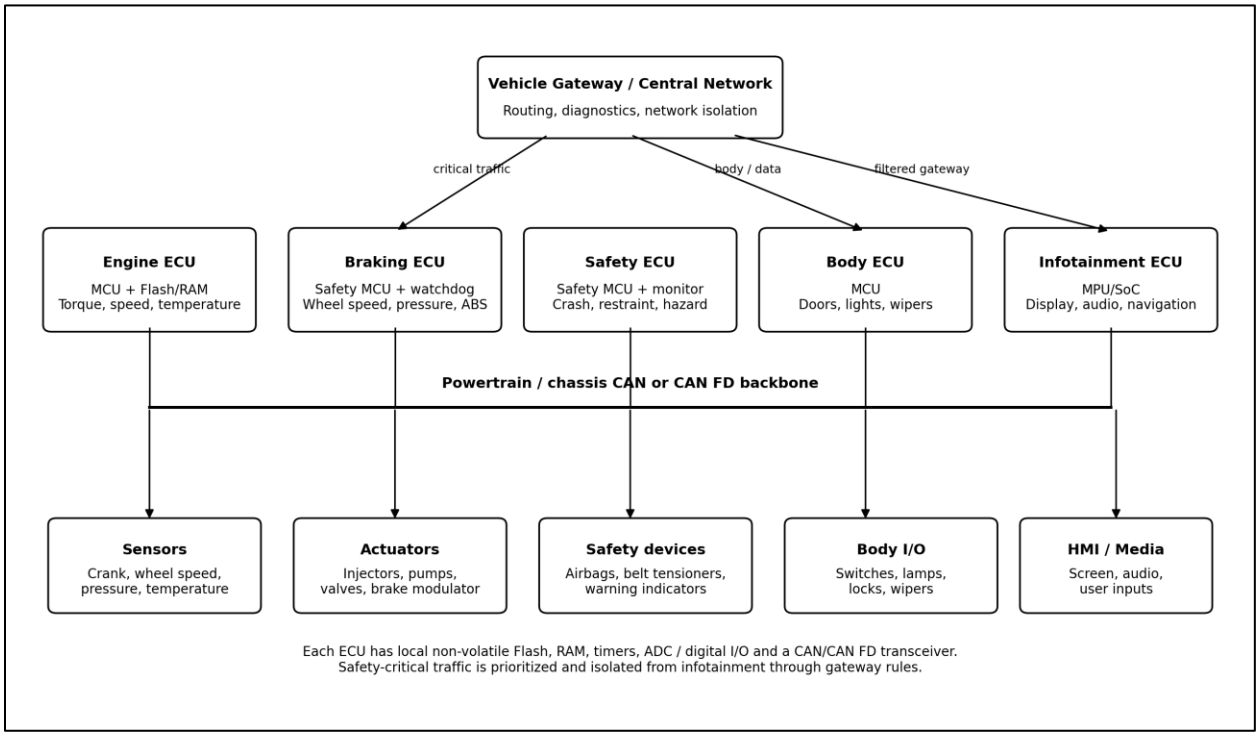


Figure 1. Proposed distributed automotive embedded-system architecture.

4.2 ECU hardware organization

Each control ECU contains an automotive MCU, program Flash, RAM, clock/timer resources, watchdog, power conditioning, input protection, sensor interfaces, actuator drivers and a CAN/CAN FD controller/transceiver as required. The infotainment unit uses an MPU/SoC, external DRAM and non-volatile storage, display/audio interfaces and a gateway-facing network interface. Memory protection, secure boot, diagnostic access control and software-update validation are part of the detailed design.

4.3 Communication paths and information exchange

Source → destination	Information	Purpose / direction
Engine ECU → Braking / Safety ECU	Engine torque, speed, propulsion status, fault status	Coordinated torque reduction and safety context
Braking ECU → Engine ECU	Brake request, wheel-slip/traction status	Coordinate torque intervention and stability functions
Braking ECU → Safety ECU	Brake status, deceleration estimate, ABS active/fault	Safety monitoring and event context
Safety ECU → Body ECU	Hazard warning / restraint status	Activate warnings and appropriate body responses
Body ECU → Gateway / other ECUs	Door state, lighting state, wake/sleep status	Vehicle-state awareness and energy management
Gateway → Infotainment	Approved speed, range, warning and status signals	Display and driver information only
Infotainment → Gateway	User-selected comfort requests, navigation context	Filtered requests; no direct safety actuator command

Sensor-to-ECU and ECU-to-actuator paths should be local wherever possible. For example, wheel-speed sensors connect directly to the braking ECU, and brake-modulator commands are generated locally. The bus distributes status and coordination data; it is not placed in the innermost fast control loop unless the control design explicitly proves the timing and safety case.

4.4 Role of the communication bus

CAN is a multi-master, message-oriented serial bus. Nodes contend for access using nondestructive bitwise arbitration on message identifiers. The bus provides frame-level error detection and retransmission behavior, while each receiving ECU decides whether a message is relevant. A gateway separates bus segments, filters traffic and prevents non-critical traffic from consuming safety-network capacity. Termination, topology, bit timing, transceiver selection and wiring must follow the applicable physical-layer requirements.

5. STAGE 4 – CLASSICAL CAN AND CAN FD ANALYSIS

5.1 Protocol comparison

5.2 Arbitration analysis

CAN uses dominant and recessive bus levels. During arbitration, each transmitter monitors the bus while sending its identifier. A node that sends a recessive bit but observes a dominant bit has lost arbitration and stops transmitting without corrupting the winning frame. The identifier with the numerically lower value generally has higher priority because it presents dominant bits earlier in the arbitration sequence.

Example: if a braking ECU transmits identifier 0x080 and a body ECU transmits 0x300 at the same time, the braking frame wins arbitration. The body ECU detects the mismatch, withdraws, and retries when the bus becomes available. The safety-critical message therefore gains access first, provided the network is not already occupied by a lower-priority frame and the bus-load analysis confirms its deadline.

Priority must be assigned by deadline and safety criticality, not by convenience. Identifiers must be unique within the network design, and high-priority messages must not be transmitted so frequently that lower-priority traffic is starved. CAN arbitration is non-preemptive once a frame has started; a high-priority message may wait for the currently transmitting frame to finish.

5.3 Payload exceeding Classical CAN

If an application message exceeds eight bytes, Classical CAN requires segmentation across multiple frames or a higher-layer transport protocol. Segmentation adds framing overhead, reassembly state, latency and failure cases. CAN FD can carry up to 64 data bytes in one frame and may reduce overhead. It is appropriate for larger diagnostic records, calibration blocks or grouped sensor data, provided all nodes on that segment support CAN FD and worst-case timing remains acceptable.

5.4 Design Decision 2 – Hybrid network selection

I recommend a hybrid CAN / CAN FD architecture for the powertrain, chassis, body and gateway domains, with Classical CAN retained where compact legacy control messages and existing hardware make it appropriate, and CAN FD used on selected segments for larger payloads**.** Braking and safety messages should use short, high-priority frames with carefully

controlled periods. CAN FD should not be introduced merely to increase nominal throughput; it should be used where payload size or bus-load analysis demonstrates a measurable benefit.

- Braking and safety segment: use CAN FD-capable controllers where required, but keep urgent control messages compact and prioritize them using identifiers. Validate worst-case latency and error behavior.
- Engine/powertrain segment: use CAN FD if the ECU set and application payloads justify it; otherwise Classical CAN remains adequate for small periodic control signals.
- Body segment: Classical CAN is usually sufficient for switches, lamps, locks and status messages.
- Infotainment: use a higher-bandwidth Ethernet-based network for media streams where needed; exchange vehicle signals through a gateway, not by flooding the control CAN bus.

Important compatibility condition: do not place legacy Classical-CAN-only nodes on a CAN FD segment if they cannot correctly tolerate FD frames. Use separate physical segments and a gateway where mixed generations of equipment must coexist.

6. STAGE 5 – FORMAL MODELING OF ELECTRONIC BRAKING

6.1 Selected function and modeling method

Electronic braking is selected because it is safety-critical, event-driven and has clearly defined operating modes. A finite-state machine (FSM), expressed as a Moore-style model, is used. In a Moore machine, outputs are associated with states, which makes the commanded operating mode explicit. Guards on transitions represent sensor validity, pedal demand, wheel slip and detected faults.

6.2 States, inputs and outputs

Element	Definition
States	MONITOR; BRAKE REQUEST; PRESSURE CONTROL; ABS REGULATION; SAFE DEGRADED
Inputs / events	Brake pedal demand; wheel speeds; pressure feedback; vehicle speed; sensor-valid flags; ECU watchdog; bus timeout
Outputs / actions	Hydraulic pressure command; pump/valve command; ABS status message; warning request; fault log; safe fallback request
Concurrent activities	Watchdog supervision, sensor plausibility checks and network timeout monitoring execute alongside the main control state machine

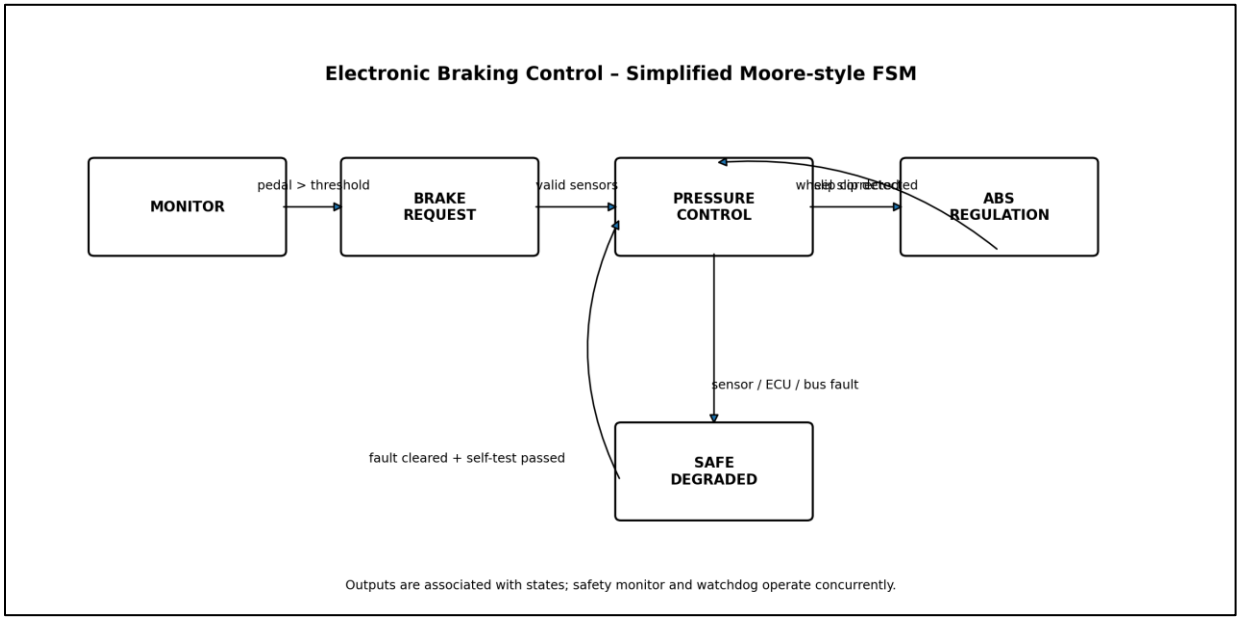


Figure 2. Simplified Moore-style finite-state machine for electronic braking.

6.3 Transition logic

Current state	Condition / event	Next state	Action
MONITOR	Brake pedal exceeds calibrated threshold and signals valid	BRAKE REQUEST	Calculate requested deceleration
BRAKE REQUEST	Wheel-speed and pressure signals plausible	PRESSURE CONTROL	Command target pressure
PRESSURE CONTROL	Wheel slip exceeds calibrated threshold	ABS REGULATION	Modulate pressure to control slip
ABS REGULATION	Slip returns to controlled range	PRESSURE CONTROL	Resume pressure tracking
Any active state	Critical sensor, ECU or communication fault	SAFE DEGRADED	Set fault flag; use defined fallback; warn driver
SAFE DEGRADED	Fault cleared and self-test passes	MONITOR	Re-enable normal control under safety policy

The actual braking fallback must be defined by the vehicle safety concept. The simplified model does not prescribe a universal hydraulic response; it identifies the need for a validated degraded

mode. Transition guards require hysteresis and plausibility limits to prevent rapid state oscillation.

6.4 Design Decision 3 – Why FSM is appropriate

An FSM is appropriate because braking control has a finite set of operational modes, discrete events and explicit transitions. It is straightforward to review, simulate, test for transition coverage and trace to requirements. A Petri net could better represent concurrency across several independent subsystems, while SysML could describe broader system interactions. For this focused control function, an FSM provides the clearest balance of rigor and readability. Concurrent watchdog and diagnostic tasks are stated separately and must be verified with scheduling analysis.

7. STAGE 6 – VERIFICATION AND CRITICAL EVALUATION

7.1 Real-time performance and bus-load verification approach

A defensible timing analysis must calculate worst-case response time for every periodic and event-driven message. Inputs include nominal bit rate, frame length, bit stuffing, inter-frame spacing, message period, identifier priority, release jitter, retransmissions and maximum bus occupancy. CAN arbitration is fixed-priority and non-preemptive; therefore, response-time analysis must include blocking by one lower-priority frame and interference from higher-priority messages.

For a first-order illustration, assume a 500 kbit/s Classical CAN segment with an estimated 130 transmitted bits per 8-byte frame including overhead and stuffing allowance. One frame occupies approximately 130 / 500,000 = 0.00026 s, or 0.26 ms. If 20 such frames are sent every 10 ms, their approximate utilization is 20 × 0.26 / 10 = 52%. This estimate excludes retransmissions and detailed frame-specific bit stuffing, so it is not a final acceptance result. A design target such as keeping sustained utilization below approximately 50–60% can provide margin, but the correct limit depends on deadlines and worst-case traffic.

For CAN FD, the arbitration portion remains at the nominal rate while the data phase may run faster with BRS. Exact frame time depends on payload length, CRC, bit stuffing, controller timing and transceiver capability. Therefore, the project should use a frame-level timing calculator or network simulation and then confirm results on a representative bus.

7.2 Reliability, error handling and fault containment

- CAN error detection includes bit monitoring, stuff checks, frame checks, CRC and acknowledgement checks. Transmit and receive error counters support error-active, error-passive and bus-off fault confinement.
- A bus-off ECU must stop participating in transmission until the defined recovery procedure is satisfied. Recovery must be supervised; repeated automatic recovery without diagnosis can hide persistent faults.
- Use end-of-line termination at both ends of each high-speed CAN segment, correct cable topology, controlled stub lengths, suitable grounding and automotive-qualified transceivers.
- Use message counters, timeout supervision, plausibility checks and application-level end-to-end protection where safety analysis requires detection of stale, repeated, lost or corrupted application data.
- Use independent watchdogs, brownout detection, safe output defaults, protected power supplies and diagnostic trouble codes. Critical actuators should not rely solely on a remote ECU message.

7.3 Failure scenarios and mitigation

Failure scenario	System effect	Mitigation	Reliable operation strategy
1. Braking ECU loses power or enters a software lockup	ABS control and network status may be lost; braking performance may be degraded	Independent watchdog and power supervision; fail-safe actuator strategy; redundant sensing/monitoring where justified; fault reporting	Base hydraulic braking remains available according to the vehicle design; warning is issued and the vehicle enters a validated degraded mode
2. CAN bus short, open circuit or persistent noise	Messages are delayed or lost; nodes may enter error-passive/bus-off	Segmented buses, correct termination, wiring diagnostics, error counters, bus guardian/gateway isolation where appropriate	Local control loops continue using local sensors; affected domain is isolated and non-essential functions are shed
3. Wheel-speed sensor stuck or implausible	ABS may calculate incorrect slip and pressure modulation	Cross-check wheel speeds, range/rate checks, redundant or diverse estimates, diagnostic timeout	Disable only the affected advanced function according to safety policy; retain base braking and warn driver
4. Gateway failure or compromised infotainment traffic	Cross-domain information may be unavailable or excessive traffic may be introduced	Gateway watchdog, message allow-list, rate limiting, secure boot, network segmentation and fail-silent behavior	Safety ECUs continue local control; loss of display/status is tolerated without allowing infotainment to command safety actuators

7.4 Redundancy and safety measures

Redundancy should be applied according to hazard analysis rather than added indiscriminately. Examples include plausibility comparison between independent wheel-speed channels, dual-channel pedal sensing, redundant power feeds for selected safety controllers, independent watchdogs and redundant communication paths only where the safety concept requires them. Redundant components must avoid common-cause failures through appropriate physical, electrical and software independence.

7.5 Scalability, processing, power and memory

The architecture scales by adding nodes to suitable segments, reserving identifier ranges, defining gateway routing rules and measuring bus load before integration. Additional sensors or diagnostic payloads may justify CAN FD, while camera/radar raw data or media streams generally require automotive Ethernet or another high-bandwidth link. MCU memory and CPU utilization must be measured under worst-case execution, including diagnostics and fault handling. Sleep/wake strategies, transceiver standby modes and ECU power budgets are required to meet vehicle energy targets.

7.6 Verification plan

Verification activity	Method / evidence	Acceptance focus
Requirements traceability	Map each requirement to ECU, message, software function and test	No unallocated safety or functional requirement
Timing analysis	Worst-case execution-time analysis and CAN response-time/bus-load analysis	All deadlines met with documented margin
Protocol conformance	Controller/transceiver tests, bit timing and error injection	Correct framing, arbitration and fault confinement
Fault injection	Disconnect sensors, force timeouts, bus-off, brownout and watchdog reset	Defined safe/degraded behavior; no uncontrolled actuation
Hardware-in-loop	Replay sensor profiles and vehicle scenarios against ECU hardware	Control response, state transitions and diagnostics correct
Environmental/EMC testing	Temperature, supply transients, vibration and electromagnetic tests	Operation within automotive component and vehicle limits
Security and update tests	Gateway access control, secure boot and update interruption tests	Unauthorized commands blocked; recovery from interrupted update

8. FINAL ENGINEERING DECISION – DEPLOYMENT

Deployment decision: YES – proceed to detailed engineering and prototype validation, but do not release directly to production on the basis of this high-level design alone.

The proposed architecture meets the stated functional intent by assigning dedicated responsibilities to five ECUs, selecting deterministic MCUs for control and an SoC for infotainment, prioritizing safety traffic and isolating non-critical domains. A hybrid CAN/CAN FD approach provides a practical migration path: compact control traffic remains efficient, while selected larger payloads can use CAN FD after compatibility and schedulability checks. The braking FSM makes operating states and fault transitions explicit. Local control, watchdogs, bus segmentation, gateway filtering and fault diagnostics improve resilience.

The principal limitation is that exact vehicle-specific deadlines, bus loads, safety goals, hardware part numbers and redundancy levels have not been supplied. The next engineering phase must complete hazard analysis, safety requirements allocation, quantitative response-time analysis, CAN FD compatibility review, software architecture, cybersecurity threat analysis and vehicle-level testing. Production deployment is conditional on successful completion of these activities and applicable automotive safety and quality processes.

8.1 Required next-stage actions

1. Define vehicle-level safety goals and allocate requirements to each ECU and communication path.
2. Select exact MCU/SoC devices using worst-case CPU, memory, peripheral, temperature, lifecycle and safety evidence.
3. Create a complete message database with identifiers, payloads, periods, deadlines, priorities and timeout behavior.
4. Perform worst-case CAN/CAN FD bus-load and response-time analysis, including error/retransmission assumptions.
5. Validate braking state-machine transitions through model-based tests, hardware-in-loop tests and fault injection.
6. Complete gateway security, software update, diagnostics, EMC, environmental and vehicle integration validation.

9. CONCLUSION

This report developed a distributed automotive embedded architecture based on explicit system requirements and engineering trade-offs. Dedicated MCUs are recommended for deterministic control ECUs, while an MPU/SoC is reserved for infotainment. A segmented network and controlled gateway separate safety-critical traffic from non-critical applications. Classical CAN remains suitable for compact control messages, while CAN FD is justified for selected larger-payload segments when all connected nodes are compatible and timing analysis supports its use. An FSM provides a clear behavioral model for electronic braking, including a safe-degraded state. The evaluation identifies realistic ECU, sensor, bus and gateway failure modes and proposes layered mitigation.

Further work should replace illustrative timing assumptions with the actual network database and measured execution times, complete formal safety and cybersecurity analyses, and validate the design on hardware-in-loop and vehicle platforms. These steps are essential before production release.

REFERENCES

Bosch. (1991). CAN specification version 2.0. Robert Bosch GmbH.

International Organization for Standardization. (2015). ISO 11898-1:2015: Road vehicles—Controller area network (CAN)—Part 1: Data link layer and physical coding sublayer. ISO.

International Organization for Standardization. (2016). ISO 11898-2:2016: Road vehicles—Controller area network (CAN)—Part 2: High-speed medium access unit. ISO.

International Organization for Standardization. (2018). ISO 26262:2018: Road vehicles—Functional safety (Parts 1–12). ISO.

International Organization for Standardization. (2021). ISO 21434:2021: Road vehicles—Cybersecurity engineering. ISO.

CiA. (n.d.). CAN FD—The basic idea. CAN in Automation. https://www.can-cia.org/can-knowledge/can-fd-the-basic-idea/

Robert Bosch GmbH. (2012). CAN with flexible data-rate (CAN FD). In 13th International CAN Conference (iCC).

Barr, M., & Massa, A. (2006). Programming embedded systems (2nd ed.). O'Reilly Media.

APPENDIX A – DESIGN ASSUMPTIONS AND LIMITATIONS

- The automobile is assumed to use a 12 V low-voltage electrical system; hybrid/electric high-voltage architecture is outside this conceptual scope.
- CAN identifiers, message periods, frame lengths and bit rates in the examples are illustrative and must be replaced with vehicle-specific values.
- The braking FSM is a simplified educational model, not a complete brake controller specification.
- Safety integrity levels, diagnostic coverage targets and hardware redundancy are not assigned because the vehicle hazard analysis is not provided.
- CAN FD deployment requires FD-capable controllers and compatible physical network design on each FD segment.

APPENDIX B – ABBREVIATIONS

Abbreviation	**Meaning**
ABS	Anti-lock Braking System
CAN	Controller Area Network
CAN FD	Controller Area Network Flexible Data-rate
ECU	Electronic Control Unit
EMC	Electromagnetic Compatibility
FSM	Finite-State Machine
MCU	Microcontroller Unit
MPU	Microprocessor Unit
SoC	System on Chip
RAM	Random Access Memory
ROM / Flash	Non-volatile program memory
CRC	Cyclic Redundancy Check